

Images, mosaïque et Canvas

Du fichier envoyé à l'image autorisée, puis de la galerie du projet au PNG produit dans le navigateur.

Un upload est une suite d'étapes

Formulaire multipart ? fichier temporaire ? contrôles PHP ? stockage autorisé ? ligne MySQL liée au projet

Le formulaire utilise POST, le jeton CSRF et multipart/form-data. PHP vérifie notamment le résultat de l'envoi, la taille, le type MIME avec Fileinfo, le décodage et les dimensions.

Une extension comme .jpg ne prouve pas que le contenu est une image JPEG.

Le nom d'origine sert à la légende ; un nom de stockage généré évite les collisions et n'est pas choisi par le navigateur. Le fichier reste dans storage/projects/{id}/ ; la base conserve l'id du projet, le nom de stockage, le type et les métadonnées nécessaires.

Point d'accès image contrôlé

Le navigateur demande l'image par son identifiant, pas avec un chemin disque libre.

image.php retrouve la ligne SQL, confirme que l'image appartient au projet demandé, valide le nom de stockage et le type réel, puis renvoie l'image. Pour ?thumb=1, il produit une miniature selon les règles définies.

Dans projet.php, la requête WHERE project_id = :project_id limite la galerie au projet ouvert. Chaque entrée est transmise au JavaScript avec une URL contrôlée de type image.php?id=...&project_id=...&thumb=1. Aucun chemin privé du serveur ne doit être donné au navigateur.

Rôles dans le rendu de mosaïque

| Élément | | Rôle |

| PHP / json_encode() | | Transmet au navigateur des données d'image sûres et structurées. |

| renderPinterest() | | Répartit les images en tuiles dans l'aperçu HTML. |

| CSS | | Présente et met à l'échelle l'aperçu à l'écran. |

| Canvas / exportMosaicToPng() | | Dessine les images dans un canevas puis génère un fichier PNG. |

La valeur appelée seed initialise une suite pseudo-aléatoire. Une valeur identique peut aider à reproduire les choix de placement ; si elle est vide, le code doit décider et conserver quelle seed effective a été utilisée.

Aperçu, réglages et export

Essayer des options pour l'aperçu est différent de les enregistrer durablement. Une URL GET convient à une demande d'aperçu. Pour mémoriser les réglages d'un projet, le navigateur doit envoyer une action POST avec le jeton CSRF ; PHP valide les valeurs avant de les stocker, répond en JSON, et la page relit ces valeurs à la prochaine visite.

La marge agrandit le PNG autour de la mosaïque. Pour une image de base $w \times h$ avec marge m , la taille exportée est $(w + 2m) \times (h + 2m)$. Il faut vérifier la taille et le fond du fichier téléchargé, pas seulement l'apparence de l'aperçu.

État observé dans le dépôt lors de la revue : les images du projet alimentent le rendu et l'export ; les réglages d'aperçu viennent actuellement des paramètres GET. La persistance des réglages par projet est un point restant à implémenter ou à vérifier. Ce constat peut changer quand le code évolue.

Petite recette d'essais

- Ouvrir deux projets aux images différentes : aucune image ne doit se mélanger.
- Tester une image seule : elle peut remplir plusieurs tuiles.

- Tester sans image : message utile, aucun export trompeur.
- Essayer une valeur limite et une valeur invalide : pas de détails techniques exposés.
- Enregistrer des réglages, quitter la page, revenir : vérifier les valeurs et la case de transparence décochée.
- Ouvrir le PNG : dimensions, marge, fond et contenu attendus.
- Simuler un échec d'enregistrement : aperçu utilisable, message d'échec, aucun faux « enregistré ».

Phrase pour l'expliquer

« La base fournit les images du projet. Le point d'accès vérifie leur appartenance avant de les servir, y compris en miniature. JavaScript les place dans l'aperçu et Canvas dessine le PNG. Les réglages durables doivent passer par POST protégé et être relus depuis le serveur.

»

À relier aux missions 6 et 7. Ce support explique les notions ; les cases de validation de la mission restent à vérifier dans le navigateur et avec l'accompagnant.